Name of the Student	**S. Yuva Guru Mani Varma**
Reg. No	**192425005**
GitHub Link	**https://github.com/YuvaGuruManiVarmaS-018/MLA0405-192425005**

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

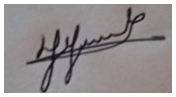
COURSE NAME: Deep Learning **COURSE CODE: MLA04**
SLOT :

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10% **Duration: 90 minutes**
Assessment Tool Weightage: 40% **Mark : 25**

Code of Conduct:
I **S. Yuva Guru Mani Varma / 192425005** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **Name of the student: S. Yuva Guru Mani Varma**

Criteria	Understanding of Industry Problem **(2)**	Application of Engineering Concepts **(2.5)**	Solution Design **(2.5)**	Use of Tools **(2)**	Analysis ***(1)***	**Total (10)**
Weightage	***20 %***	***25%***	***25%***	***20 %***	***10 %***	**100%**
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS: 05 Total Marks:25

Q1. Transfer Learning

Proposed model: Use **ResNet-50** pre-trained on ImageNet.
Architecture and strategy

1. **Input:** Dermoscopic image, resized to 224 × 224 pixels.
2. **Pre-trained CNN:** Load ResNet-50 with ImageNet weights.
3. **Initial layers:** Freeze the convolutional layers because they learn general features such as edges, textures, shapes, and color patterns.
4. **Higher layers:** Unfreeze the last 1–2 residual blocks because these layers learn more task-specific features.
5. **Classification head:** Remove the original ImageNet classifier and add:
    - Global Average Pooling
    - Fully Connected/Dense layer
    - Dropout
    - Output layer with **2 classes** (benign and malignant) or multiple skin-lesion classes.
6. **Training:** First train only the new classification head. Then fine-tune the unfrozen upper layers using a small learning rate.

Why ResNet-50?
ResNet-50 uses **residual/skip connections**, which allow deep networks to learn effectively without severe vanishing-gradient problems. Its ImageNet-trained features can be reused for dermoscopic images.
Advantages

- Requires fewer labeled medical images.
- Reduces training time and computational cost.
- Frozen layers prevent overfitting.
- Fine-tuning higher layers adapts the model to skin-lesion patterns.
- Data augmentation such as rotation, flipping, cropping, and brightness variation improves generalization.
- The combination of transfer learning and fine-tuning generally provides better classification performance than training a CNN from scratch.

Overall: The proposed ResNet-50 transfer-learning model learns general visual features from ImageNet and adapts its deeper layers to skin-cancer characteristics, improving accuracy and generalization with limited medical data.

Q2. Dense Net and Pixel Net

Proposed architecture
Input Image → DenseNet Encoder → Feature Fusion → PixelNet Decoder → Pixel-wise **Classification:**

1. DenseNet Encoder
Use **DenseNet-121** as the feature-extraction network.

DenseNet connects each layer to subsequent layers:
Layer 1 → Layer 2 → Layer 3 → Layer 4 → ...
Each layer receives feature maps from previous layers. This creates extensive **feature reuse**.
The encoder extracts:

- Low-level edges and textures
- Vehicle and pedestrian shapes
- Road boundaries
- Traffic-sign patterns
- High-level semantic features

2. PixelNet Segmentation Module
The extracted DenseNet features are passed to a PixelNet-style pixel-level prediction network.
The decoder:

- Upsamples the feature maps.
- Combines high-level semantic information with lower-level spatial features.
- Produces a prediction for every pixel.
- Uses **Softmax** to classify each pixel into road, pedestrian, vehicle, traffic sign, background, etc.

3. Skip Connections
Features from earlier DenseNet layers can be connected to corresponding decoder layers.
This helps preserve:

- Object boundaries
- Small traffic signs
- Thin pedestrian structures
- Fine road details

How it improves the system

Feature	**Improvement**
Dense Net	Reuses features from earlier layers
Dense connections	Improves gradient flow and feature propagation
Pixel Net	Performs pixel-level classification
Skip connections	Preserve spatial details and boundaries
Feature reuse	Reduces the need to learn duplicate features
Compact architecture	Can require fewer parameters than some traditional deep CNNs
Multi-scale features	Helps detect both large vehicles and small traffic signs

Overall architecture
Road Image
↓
DenseNet-121 Encoder
↓
Dense Feature Maps
↓
Feature Fusion + Skip Connections
↓
PixelNet Decoder
↓
Upsampling
↓

Pixel-wise Softmax
↓
Road / Pedestrian / Vehicle / Traffic Sign / Background

Conclusion: Dense Net provides efficient feature extraction through dense feature reuse, while Pixel Net performs detailed pixel-level classification. Combining them improves segmentation accuracy, preserves object boundaries, reduces redundant feature learning, and provides an efficient solution for automated smart-city road-scene analysis.

Q3. Bidirectional RNN and Encoder–Decoder

Proposed Algorithm:

A suitable solution is an **Encoder–Decoder (Seq2Seq) model with Bidirectional RNNs** for multilingual translation.

Architecture

Input Sentence
↓
Word/Token Embedding
↓
Bidirectional RNN Encoder
↓
Context Vector
↓
RNN/LSTM Decoder
↓
Attention Mechanism
↓
Softmax Output
↓
Translated Sentence

1. Encoder

The input sentence is converted into word embeddings. A **Bidirectional RNN (Bi-RNN)** processes the sentence in two directions:

- Forward: beginning → end
- Backward: end → beginning

The two hidden states are combined to create a richer representation of each word.

2. Decoder

The decoder receives the encoded context and generates the translated sentence **one word at a time**.

For example:

Input:

How are you?

Output:

¿Cómo estás?

The decoder uses the previously generated word and the encoder's context information to predict the next word.

3. Attention

An attention mechanism can be added so that the decoder focuses on the most relevant encoder states for each output word. This is especially useful for long sentences.

Why Bidirectional RNN improves translation

- Reads the complete sentence from **both directions**.
- Captures both previous and following word information.
- Provides better contextual understanding.
- Helps resolve ambiguous words.
- Improves translation of long and complex sentences.
- Combined with attention, it allows the decoder to focus on relevant parts of the input.

Conclusion: A Bidirectional RNN encoder with an attention-based decoder provides better contextual representations than a one-directional RNN, making it suitable for multilingual chatbot translation.

Q4. BPTT and LSTM

Proposed algorithm:

A suitable system for stock-price forecasting is an LSTM-based time-series prediction model.

Architecture

Historical Stock Data
↓
Data Normalization
↓
Sequence Creation
↓
LSTM Layer
↓
Dropout Layer
↓
Dense Layer
↓
Predicted Stock Price

For example, the model can use the previous 30 days of stock features such as:

- Opening price
- Closing price
- High and low prices
- Trading volume

to predict the next day's closing price.

LSTM Structure

An LSTM contains three important gates:

1. Forget Gate – decides which old information should be removed.
2. Input Gate – decides which new information should be stored.
3. Output Gate – decides which information should be used as the current output.

The cell state carries important information through many time steps.

BPTT – Backpropagation Through Time

BPTT is used to train the LSTM.

Process:

1. Input historical sequences into the LSTM.
2. The LSTM produces a predicted stock price.
3. Calculate the error using a loss function such as Mean Squared Error (MSE).

4. The network is conceptually unrolled across time steps.
5. The error is propagated backward through these time steps.
6. Gradients are calculated for the LSTM weights.
7. An optimizer such as Adam updates the weights.
8. The process is repeated for multiple epochs.

Why LSTM is better than traditional RNN

Traditional RNN	LSTM
Has difficulty learning long-term dependencies	Can retain long-term information
More affected by vanishing gradients	Gates help control gradient flow
Basic memory mechanism	Uses cell state and gates
Less suitable for long historical sequences	Better for long time-series data
May forget important past information	Can selectively remember important information

Conclusion

LSTM is more suitable for stock-price forecasting because market data contains temporal dependencies where information from earlier time periods can influence future values. LSTM's memory cells and gates allow it to retain useful long-term information and reduce the vanishing-gradient problem. BPTT enables the model to learn these temporal relationships by propagating prediction errors backward through the sequence.

Q5. Integrated Deep Learning Solution

A suitable solution is a **Video Caption Generation System** that combines **Transfer Learning** for extracting visual features from video frames with an **LSTM Encoder–Decoder** for generating captions.

Architecture

Input Video
↓
Frame Extraction
↓
Pre-trained CNN (Transfer Learning)
↓
Visual Feature Vectors
↓
LSTM Encoder
↓
Context Representation
↓
LSTM Decoder
↓
Word-by-Word Caption Generation
↓
Final Caption

1. Transfer Learning for Visual Feature Extraction

First, the uploaded video is divided into individual frames. A pre-trained CNN such as **ResNet-50** or **InceptionV3** is used to extract visual features.

The CNN is pre-trained on a large image dataset, so its earlier layers already understand general features such as:

- Edges
- Shapes
- Textures
- Objects
- Colors

The final classification layer is removed, and the CNN is used as a **feature extractor**. The resulting feature vectors represent the important visual information in each video frame.

Example:

Video frames → CNN → [feature$_1$, feature$_2$, feature$_3$, ...]

2. LSTM Encoder

The sequence of visual feature vectors is given to an **LSTM Encoder**.

The encoder processes the frames in temporal order and learns:

- What objects are present
- What actions are occurring
- How the scene changes over time

The LSTM's memory mechanism helps preserve important information from earlier frames.

3. LSTM Decoder

The decoder generates the caption one word at a time.

For example, if the video shows a person riding a bicycle:

<START> → A → person → is → riding → a → bicycle → <END>

At every time step, the decoder uses the encoded visual information and previously generated words to predict the next word.

4. Training

During training, videos are paired with their correct captions.

The system:

1. Extracts frames from the video.
2. Extracts visual features using the pre-trained CNN.
3. Passes the feature sequence to the LSTM encoder.
4. The decoder generates a caption.
5. The generated caption is compared with the actual caption.
6. The prediction error is backpropagated to update the trainable parameters.

Cross-entropy loss can be used for word prediction.

Role of Each Component

Component	Role
Video	Provides the visual input
Frame Extraction	Converts video into a sequence of images
Pre-trained CNN	Extracts meaningful visual features
Transfer Learning	Reduces training requirements and improves feature extraction
LSTM Encoder	Understands the temporal sequence of video features
LSTM Decoder	Converts visual information into natural-language captions
Softmax	Selects the most probable next word

Why the Integrated System Works Well

- **Transfer learning** provides strong visual representations even when the company has limited training data.
- **LSTM** captures temporal relationships between video frames.
- The **Encoder–Decoder structure** converts visual information into a sequence of words.
- LSTM memory helps maintain context throughout the generated sentence.
- The system can generate captions that describe both **objects and actions** in the video.
- The complete pipeline can be trained end-to-end after the CNN feature extractor is initially frozen.

Conclusion

The combination of **Transfer Learning + CNN + LSTM Encoder–Decoder** provides an effective video-captioning system. The CNN understands **what is visible**, while the LSTM understands **how the visual information changes over time** and converts it into meaningful language. This integrated approach improves caption accuracy, reduces training time, and makes automatic video captioning practical for accessibility applications.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	20%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis